ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA THÀNH PHỐ HỒ CHÍ MINH

THỰC TẬP ĐỒ ÁN ĐA NGÀNH
HƯỚNG TRÍ TUỆ NHÂN TẠO

Báo cáo: Khảo sát cách dùng LLM để tối ưu hóa hiệu năng các ứng dụng AI hiện nay

Giảng viên phụ trách: Ths. Hoàng Lê Hải Thanh

Thực hiện: Nguyễn Ánh Dương – 2310596

Mục lục

I. Giới thiệu: 2
II. Các phương pháp tiếp cận: 3
1. Framework tối ưu hóa GPU Kernel dựa trên Minimal Executable Program [1]: 3
2. Framework đánh giá tối ưu hóa kernel của LLM cho Machine Learning (KernelBench) [3] 8
3. Tối ưu hóa dựa trên Multi-Agents (Astra Framework) [2] 10
III. SO SÁNH PHƯƠNG PHÁP: MEP VÀ ASTRA FRAMEWORK 14
1. Cách tiếp cận: 14
2. Kiến trúc: 14
3. Feedback: 14
4. Metrics: 15
IV. Kết quả thực nghiệm của MEP Framework [1] 16
V. Kết quả thực nghiệm của Astra Framework [2] 17
1. Kết quả hiệu năng tổng thể trên các kernel thực tế 17
2. Kết quả khi chạy Multi-Agents và Single-Agent 17
VI. KẾT QUẢ ĐÁNH GIÁ CODE KERNEL CỦA CÁC LLM (DỰA TRÊN KERNELBENCH) [3] 18
1. One-shot Baseline: 18
2. Error Analysis 19
3. Feedback Loop 20
VII. KẾT LUẬN 20
TÀI LIỆU THAM KHẢO 21

I. Giới thiệu:

Trong lĩnh vực HPC và AI, các GPU là quan trọng đến hiệu suất của toàn bộ hệ thống. Nhưng việc tối ưu hóa mã nguồn GPU bằng tay đòi hỏi có kinh nghiệm cao, tốn thời gian và khó khăn khi chuyển qua các kiến trúc phần cứng khác nhau. Báo cáo này tập trung khảo sát các phương pháp **tối ưu code để cải thiện performance** sử dụng Mô hình ngôn ngữ lớn (LLM). Bài khảo sát này đi sâu vào cách LLM phân tích, hiệu chỉnh và cải thiện cấu trúc mã nguồn để đạt được tốc độ thực thi tối ưu trên phần cứng mục tiêu. Bài khảo sát này khảo sát 3 framework dựa trên 3 bài báo đều được công bố trên ***Arxiv*** của ***Conrnel University,*** trong đó có 2 loại bài:

Bài hướng đến đánh giá giữa các LLM trong tối ưu code:

- Framework đánh giá tối ưu hóa kernel của LLM cho Machine Learning (KernelBench)

Bài hướng đến ứng dụng LLM vào tối ưu code:

- Framework tối ưu hóa GPU Kernel dựa trên Minimal Executable Program.
- Tối ưu hóa dựa trên Multi-Agents (Astra Framework).

Lưu ý: Tất cả các tiêu đề có đánh chỉ mục tài liệu tham khảo, thì tất cả các hình ảnh, công thức và diễn giải trong mục đó đều được chụp, trình bày lại từ chính tài liệu tham khảo.

II. Các phương pháp tiếp cận:

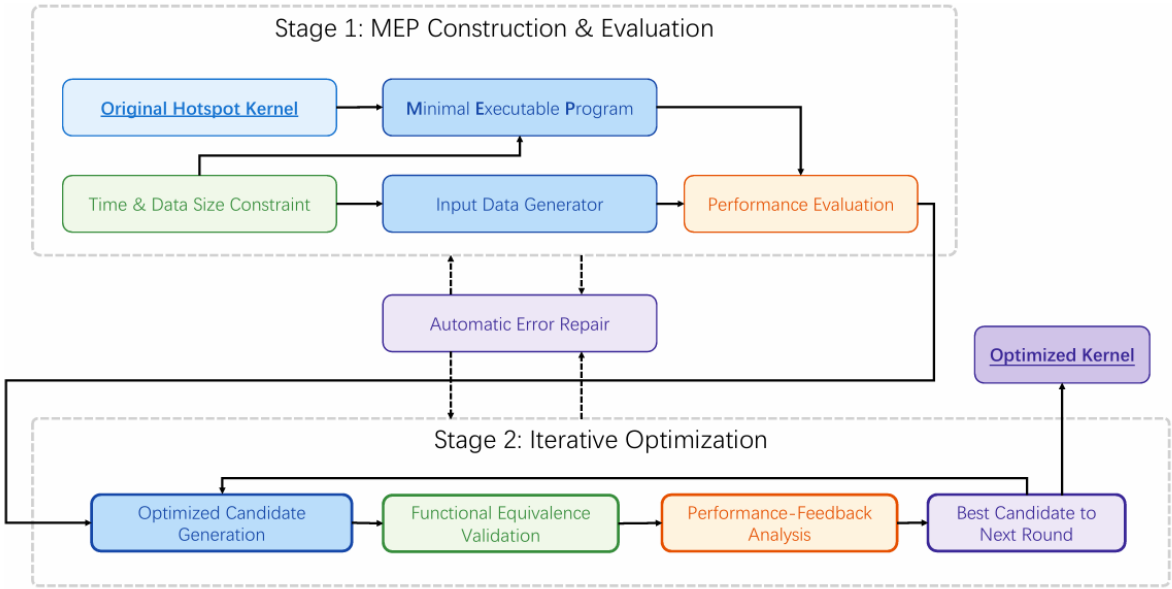
1. Framework tối ưu hóa GPU Kernel dựa trên Minimal Executable Program [1]:

Phương pháp này giải quyết bài toán tối ưu hóa các hotspot kernel (những đoạn mã tiêu tốn nhiều tài nguyên nhất) trong các ứng dụng HPC quy mô lớn mà không cần phải thực thi toàn bộ ứng dụng.

Trong framework này, nhóm tác giải đề xuất LLM tự động hoàn thiện một chương trình nhỏ từ một hotspot kernel được trích xuất riêng lẻ ra từ chương trình gốc, chương trình nhỏ đó được gọi là Minimal Executable Program (MEP). MEP là chương trình nhỏ (có chứa cái kernel gốc) có thể chạy riêng lẻ và độc lập so với chương trình gốc và việc tao ra MEP sẽ bao gồm đầy đủ các thành phần: chuẩn bị dữ liệu, cấp phát bộ nhớ, gọi kernel và xử lý kết quả. Từ đó, dựa trên MEP, LLM sẽ tiến hành tạo ra mã nguồn kernel tối ưu hơn, chạy và đánh giá để tìm ra mã nguồn kernel tối ưu nhất.

Workflow của Framework được thể hiện thông qua ***Hình 1***. Có hai giai đoạn:

- **Giai đoạn 1:** Từ Kernel gốc và ràng buộc về dữ liệu và thời gian, LLM hoàn thiện code thành MEP. Đồng thời chúng ta phải xây dựng một Input Data Generator để khớp với lại các thông số đầu vào của kernel để tiến hành chạy và đánh giá.



Hình 1. The MEP-based LLM framework cho tối ưu GPU kernel

- **Giai đoạn 2:** LLM sinh các biến thể code của kernel, từ đó kiểm tra về mặt chức năng logic của kenrel, rồi đo lường hiệu năng thực. Từ đó phản hồi kết quả để LLM cải thiện ở vòng sau.

a. Xây dựng MEP:

Ràng buộc thời gian thực thi trong MEP:

Nhóm tác giả đặt ra rằng buộc về thời gian. Cụ thể, để việc đánh giá hiệu năng mang tính thực tế và tiết kiệm, MEP phải thỏa mãn hệ thức ràng buộc sau:

$$\begin{cases} T_{ker} \geq T_{min} \\ T_{Overall} \leq T_{max} \end{cases}$$

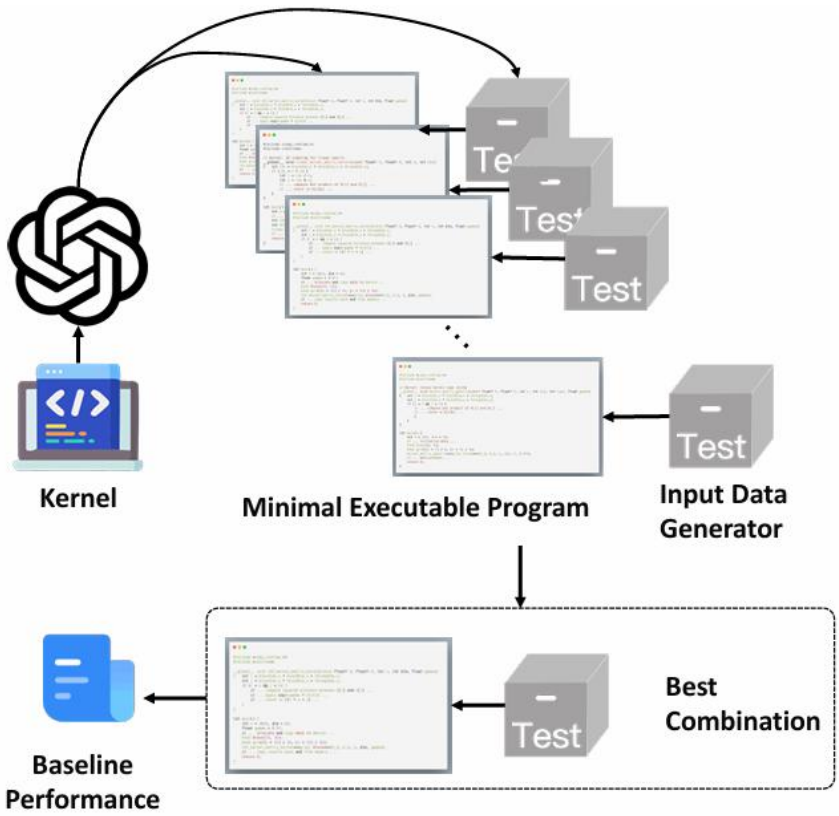
- T_{ker} : Thời gian thực thi trung bình của riêng kernel.
- T_{min} : Ngưỡng thời gian tối thiểu để đảm bảo kernel chạy đủ lâu để thấy sự khác biệt giữa các phiên bản tối ưu.
- $T_{Overall}$: Tổng thời gian chạy chương trình.
- T_{max} : Giới hạn thời gian tổng thể để đảm bảo chi phí thử nghiệm thấp.

b. Xây dựng Input Data Generator:

Nhắc lại: Input Data Generator đóng vai trò tạo ra dữ liệu input mà khớp với lại input của kernel, từ đó thực hiện chạy rồi đánh giá.

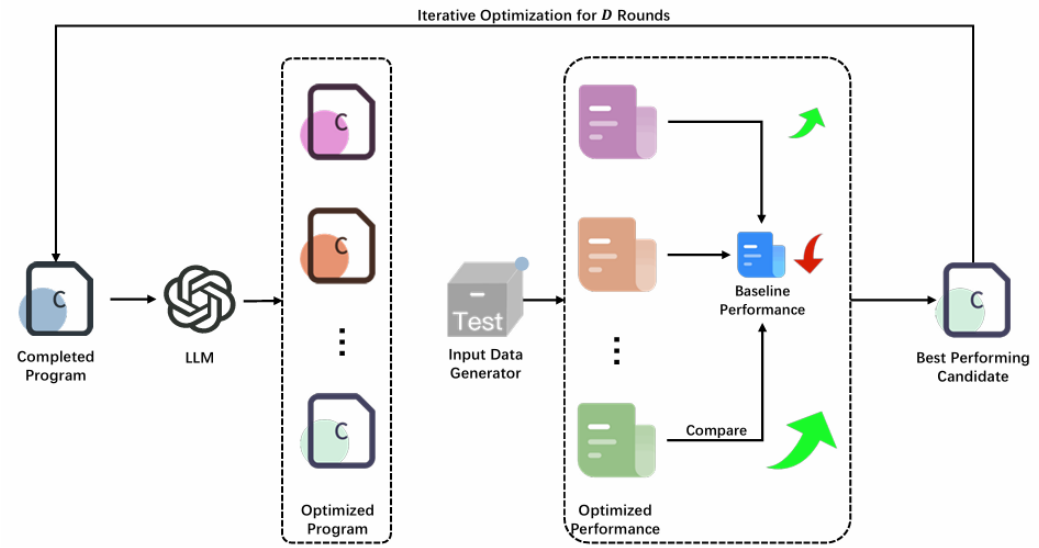
Ràng buộc giới hạn kích thước dữ liệu:

$$S_{data} \leq S_{max}$$



Hình 2. Xây dựng MEP và Inpout Data Generator và đánh giá

Giai đoạn 1 thực tế sẽ chính là việc LLM từ kernel gốc của chương trình lớn tiến hành tạo ra nhiều MEP và một Input Data Generator đi kèm. ***Hình 2*** biểu diễn chi tiết quả trình của **Giai đoạn 1**. Nó tiến hành chạy các cặp MEP và Input Data Generator để tìm ra cặp tối ưu nhất (Input Data Generator cũng đóng vai trò quan trọng vì có ảnh hưởng đến kết quả chạy, bộ dữ liệu có thể có dung lượng khác nhau) dựa trên các ràng buộc về thời gian cũa MEP và ràng buộc về giới hạn kích thước dữ liệu của Input Data Generator. Khi cặp MEP và Input Data Generator đã được chọn, chúng sẽ được dùng qua **Giai đoạn 2.**



Hình 3. Vòng lặp tối ưu về hiệu năng

Giai đoạn 2 tiếp nối với **Giai đoạn 1** sau khi chúng ta đã chọn được cặp MEP (chứa kernel gốc) và Input Data Generator tốt nhất dựa trên các ràng buộc đã nêu ở trên. Từ MEP, LLM sẽ tiến hành tạo ra các Candidate là các code kernel mới để tiến hành tối ưu riêng về hiệu năng khi chạy trên GPU. Sau đó, các Candidate này sẽ được chạy với dữ liệu đầu vào chính là của Input Data Generator, điều này giúp đảm bảo không có sự khác nhau ở dữ liệu đầu vào, thông nhất, để không làm ảnh hưởng đến kết quả hiệu năng. Dựa vào kết quả chạy, chúng ta sẽ tiến hành chọn ra code kernel tối ưu nhất, và sẽ dùng code kenrel đó cho vòng lặp tiếp theo.

Giai đoạn 2 được thể hiện một cách tổng quát như sau:
Sau khi đã có chương trình MEP ổn định ở giai đoạn trước, hệ thống bắt đầu quy trình cải tiến mã nguồn:

- Khởi tạo $K^{(0)}$: phiên bản kernel gốc làm baseline ban đầu (kernel trong MEP)
- Candidate N: Trong mỗi vòng d, dựa trên các phản ***hồi từ công cụ phân tích profiler như tỷ lệ hit cache, độ chiếm dụng tài nguyên***, LLM sẽ sinh ra tối đa N phiên bản code mới.
- Tối ưu lại: Candidate chạy nhanh nhất của vòng này sẽ trở thành baseline cho vòng kế tiếp, giúp code sẽ càng được tinh chỉnh tốt hơn.

c. Đo lường hiệu năng bằng Phương pháp Trimmed Mean:
Để loại bỏ nhiễu từ hệ thống, mỗi ứng dụng được chạy ***R*** lần. Sau đó, hệ thống loại bỏ ***k*** kết quả thấp nhất và k kết quả cao nhất trước khi tính trung bình. Công thức tính thời gian thực thi của ứng viên ***n*** trong vòng lặp ***d*** là:

$$\bar{T}_{ker}^{(d,n)} = \frac{\sum_{i=k+1}^{R-K} T_i^{(d,n)}}{R - 2k}$$

Với $R > 2k$

- R: Tổng số lần chạy thử nghiệm cho Candidate
- $T_i^{(d,n)}$: Danh sách thời gian thực thi của ***R*** lần chạy, được sắp xếp theo thứ tự tăng dần
- k: Số lượng kết quả cực biên (quá nhanh hoặc quá chậm) cần loại bỏ ở mỗi đầu
- *R-2k*: Số lượng mẫu còn lại sau khi đã cắt bỏ k mẫu thấp nhất và k mẫu cao nhất

Việc này giúp loại bỏ các giá trị ngoại lai sinh ra do nhiễu hệ thống, giúp con số hiệu năng cuối cùng đúng bản chất của thuật toán.

d. Đánh giá: lựa chọn Baseline cho vòng lặp kế tiếp:

$$C^d = \{K^{(d,n)} | 1 \leq n \leq N, FE(K^{(d,n)}, K^{(d)})\}$$

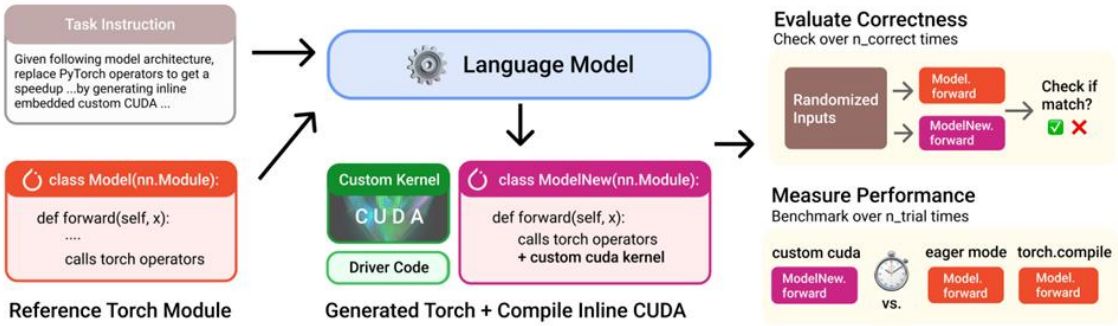
Một Candidate $K^{(d,n)}$ chỉ được chọn nếu nó thỏa mãn điều kiện **Functional Equivalence (FE)**, nghĩa là kết quả đầu ra phải hoàn toàn khớp với phiên bản baseline $K^{(d)}$ đã. Điều này để kiểm tra về tính đúng đắng logic của code kernel đó.

$$n^* = arg\,min_{1 \leq n \leq N, K^{(d,n)} \in C^d} \bar{T}_{ker}^{(d,n)}, \qquad K^{(d+1)} = K^{(d,n^*)}$$

n^*: Là chỉ số của ứng viên có thời gian thực thi trung bình thấp nhất đươc chọn ra dựa vào tập C^d chứa các Candidate thỏa FE trước đó. Phiên bản tối ưu nhất này $K^{(d,n^*)}$ sẽ trở thành baseline mới $K^{(d+1)}$ cho vòng lặp tiếp theo. Quá trình này giúp hệ thống tiến hóa mã nguồn một cách liên tục và hội tụ về điểm tối ưu.

2. Framework đánh giá tối ưu hóa kernel của LLM cho Machine Learning (KernelBench) [3]

Nghiên cứu KernelBench tiếp cận bài toán tối ưu hóa mã nguồn trực tiếp trong môi trường phát triển AI phổ biến nhất hiện nay là PyTorch. Phương pháp của framework này tập trung vào việc tự động hóa quy trình viết các CUDA kernel tùy chỉnh để thay thế các toán tử mặc định của PyTorch.



Hình 4. Overview về workflow của KernelBench

a. Mục tiêu tối ưu:

- **Task Input:** LLM nhận được một đoạn mã nguồn tham chiếu viết bằng PyTorch. Đoạn mã này chứa một class tên là ***Model*** kế thừa từ *torch.nn.Module*, bao gồm các hàm chuẩn như *__init__* và *forward()* chứa các toán tử gốc. Hệ thống cũng cung cấp sẵn các hàm *get_inputs()* và *get_init_inputs()* để xác định chính xác kích thước và kiểu dữ liệu của tensor, vì kernel tối ưu phụ thuộc rất lớn vào các yếu tố này.
- **Task Output:** LLM phải trả về một lớp ***ModelNew*** cũng kế thừa từ *torch.nn.Module*. Điểm mấu chốt là LLM sẽ sử dụng tính năng ***CUDA-C extension*** của PyTorch để tự động viết và sẽ inline compilation các đoạn mã C++/CUDA tùy chỉnh ngay bên trong hàm *forward()*.
- **Lưu ý:** LLM sẽ có quyền quyết định toán tử nào cần được tối ưu hóa và nên sử dụng kỹ thuật nào, ví dụ như là fusion hoặc tiling.

b. Phân cấp độ phức tạp của code:

Nhóm tác giả chia 250 bài toán AI thành 3 level khác nhau để đánh giá toàn diện khả năng của LLM:

Cấp độ 1 - Single primitive operation.: Tối ưu hóa các khối xây dựng cơ bản của AI như nhân ma trận, tích chập, hàm kích hoạt hoặc hàm mất mát.

Cấp độ 2 - Operator sequences.: Bao gồm các đoạn mã có nhiều toán tử thực thi liên tiếp. Ví dụ ta có một lớp convolution đi kèm với ReLU và bias. Mục tiêu ở đây là *khảo sát khả năng nhận diện và thực hiện gộp các toán tử lại thành một kernel duy nhất để giảm độ trễ truy cập bộ nhớ* của LLM.

Cấp độ 3 - Full ML architectures: Tối ưu hóa toàn bộ một kiến trúc trong Deep Learning hoặc Machine Learing như các mô hình nổi tiếng AlexNet hay MiniGPT.

c. Đo lường và đánh giá:

Vì KernelBench không có ground truth cho một kernel tối ưu nhất, framework đánh giá tự động dựa trên kết quả chạy thực tế của chương trình:

- **Correctness:** Mã nguồn **ModelNew** do LLM sinh ra sẽ được chạy thử với 5 tập tensor đầu vào ngẫu nhiên. Kết quả đầu ra output tensor phải khớp với giá trị và kích thước so với mã **Model** gốc.

- **Performance:** Đo lường thời gian chạy thực tế giữa bản gốc và bản do LLM tạo ra sau khi lặp lại nhiều lần để loại bỏ nhiễu.
  Để kết hợp **Correctness** và **Performance**, tác giả đề xuất một thước đo mới gọi là $fast_p$:

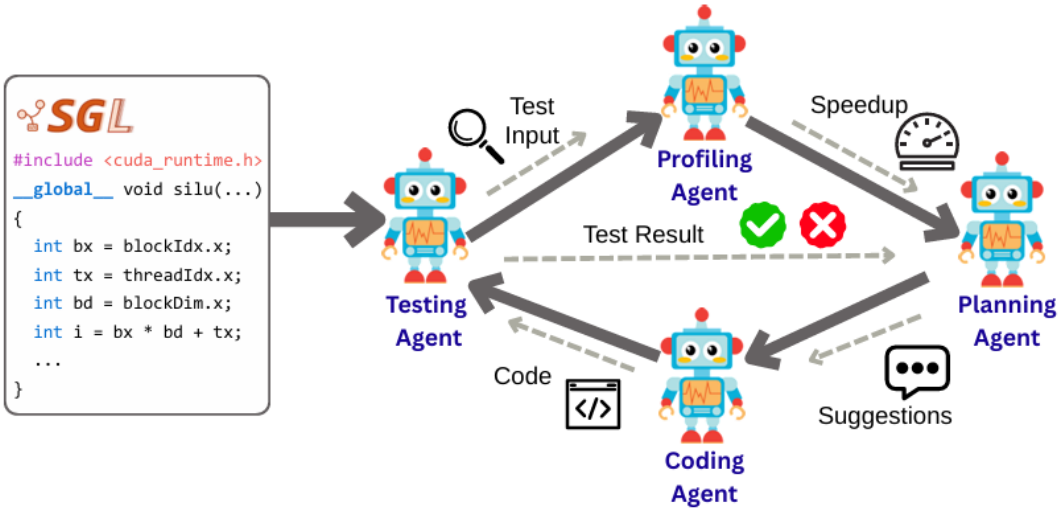
$$fast_p = \frac{1}{N}\sum_{i=1}^{N} 1(correct_i \wedge \{speedup_i > p\})$$

Trong đó:

- ***N*** là tổng số bài toán được đánh giá.
- **1** là hàm trả về 1 nếu điều kiện đúng, 0 nếu sai.
- $correct_i$ là code sinh ra chạy không lỗi và cho kết quả toán học chính xác.
- $speedup_i > p$ là điều kiện kernel mới sinh ra phải có tốc độ nhanh hơn kernel gốc lớn hơn một ngưỡng ***p*** lần.

3. Tối ưu hóa dựa trên Multi-Agents (Astra Framework) [2]

Nghiên cứu về framework Astra giải quyết một bài toán thực tế trong môi trường doanh nghiệp bằng cách tối ưu hóa các CUDA kernel đã được con người viết sẵn. Cụ thể, framework này trích xuất mã nguồn từ SGLang là một nền tảng phục vụ LLM quy mô lớn và dùng hệ thống nhiều Agent LLM để tối ưu hiệu năng phần cứng.



Hình 5. Multi-Agent hoặc động trong framework Astra

a. Multi-Agent Roles:

Framework Astra không dùng một LLM mà dùng 4 Agents chuyên biệt:

Testing Agent: Tạo ra các bộ dữ liệu đầu vào là test cases đa dạng từ kernel gốc và chịu trách nhiệm kiểm tra tính đúng đắn của các phiên bản kernel mới.
Profiling Agent: Chạy thử các kernel trên phần cứng GPU thực tế để đo lường thời gian thực thi, từ đó cung cấp phản hồi về hiệu năng.
Planning Agent: Tổng hợp các tín hiệu về tính đúng đắn và thời gian chạy từ hai Agent trên, sau đó đề ra các chiến lược tối ưu hóa cụ thể, ví dụ như thay đổi cách truy cập bộ nhớ, cấu trúc lại vòng lặp.
Coding Agent: Nhận các đề xuất từ Planning Agent để chỉnh sửa và sinh ra phiên bản code CUDA mới.

b. Công thức đo lường:

Correctness: Code tối ưu $\boldsymbol{S'}$ phải trả về kết quả tương đương với mã nguồn gốc $\boldsymbol{S}$ trên tập dữ liệu kiểm thử $T = \{(x_i, y_i)\}_{i=1}^{m}$. Để xử lý các sai số dấu phẩy động trong tính toán GPU, hệ thống cho phép một khoảng dung sai $\varepsilon \geq 0$. **Correctness** được tính bằng:

$$max_{1 \leq i \leq m} d(S'(x_i), y_i) \leq \varepsilon$$

Với $\boldsymbol{d}$ là hàm tính khoảng cách giữa đầu ra của kernel mới và đầu ra chuẩn y_i

Performance: Dùng Geometric Mean giữa các tỷ lệ tăng tốc trên toàn bộ tập kiểm thử. Công thức này được sử dụng vì nó cho biết các tỷ lệ, đối xứng giữa việc tăng tốc hoặc giảm tốc, và giảm thiểu tác động của các kết quả ngoại lại.

$$\sigma_T = \left(\prod_{i=1}^{m} \frac{\tau(S, x_i)}{\tau(S', x_i)} \right)^{1/m}$$

Trong đó, $\tau(S, x_i)$ và $\tau(S', x_i)$ lần lượt là thời gian thực thi của kernel gốc và kernel tối ưu trên cùng một đầu vào x_i.

c. Iterative Workflow:

Algorithm 1 Multi-Agent CUDA Optimization

Input: Baseline CUDA code S_0, number of rounds R
Define: `TestingAgent` ▷ Generate or run tests.
`ProfilingAgent` ▷ Measure performance.
`PlanningAgent` ▷ Propose suggestions given correctness and performance signals.
`CodingAgent` ▷ Apply suggestions to previous code S_{prev}
`Log` ▷ List of $(round, code, correctness, performance)$ for all iterations.
`Test suite` T ▷ Tests generated by `TestingAgent`.
Output: `Log`

```
T ← TestingAgent.GenerateTests(S_0)                          ▷ Initialization
perf_0 ← ProfilingAgent.Profile(S_0, T)
Log ← []
Append(Log, (0, S_0, True, perf_0))
S_prev ← S_0
pass_prev ← True
perf_prev ← perf_0
for r ← 1 to R do                                            ▷ Iterative optimization starts
    suggestions ← PlanningAgent.Suggest(S_prev, pass_prev, perf_prev)
    S_new ← CodingAgent.Apply(S_prev, suggestions)
    pass_new ← TestingAgent.Validate(S_new, T)
    perf_new ← ProfilingAgent.Profile(S_new, T)
    Append(Log, (r, S_new, pass_new, perf_new))
    S_prev ← S_new
    pass_prev ← pass_new
    perf_prev ← perf_new
return Log
```

Hình 6. Thuật toán của Astra dùng nhiều Agent để tối ưu CUDA

Phân tích về cách thuật toán hoạt động:

- ***Input và Output***: Thuật toán nhận vào mã nguồn CUDA gốc làm điểm chuẩn, ký hiệu là $\boldsymbol{S_0}$, và số lượng vòng lặp tối ưu hóa tối đa $\boldsymbol{R}$. Trả về một tệp Log chứa toàn bộ lịch sử các phiên bản mã nguồn đã sinh ra. Tệp Log này lưu trữ danh sách các bộ giá trị tuples gồm: *(round, code, correctness, performance)* cho từng vòng lặp.

- ***Initialization***: hệ thống thiết lập môi trường và đo lường
  - TestingAgent nhận mã gốc $\boldsymbol{S_0}$ và tự động sinh ra một tập dữ liệu kiểm thử T (GenerateTests)
  - ProfilingAgent chạy mã gốc $\boldsymbol{S_0}$ trên tập T để lấy dữ liệu hiệu năng ban đầu, gọi là $per\ f_0$ (Profile)
  - Hệ thống ghi lại trạng thái vòng 0 vào Log (mã $\boldsymbol{S_0}$, tính đúng đắn là True, hiệu năng $per\ f_0$)
  - Gán mã gốc và các thông số này vào các biến tạm để chuẩn bị cho vòng lặp đầu tiên.

- ***Iterative Optimization***:
  Thuật toán sẽ chạy lặp lại từ vòng $r = 1$ cho đến R. Tại mỗi vòng r:

  - **Suggest:** PlanningAgent nhận vào mã nguồn của vòng trước S_{prev}, thông tin mã đó chạy có đúng không $pass_{prev}$và hiệu năng ra sao $per\ f_{prev}$. Dựa trên tín hiệu này, PlanningAgent đề xuất các phương án sửa đổi là biến suggestions.
  - CodingAgent nhận lại mã cũ S_{prev} và các gợi ý suggestions từ Planning Agent để áp dụng, từ đó viết ra một phiên bản mã CUDA hoàn toàn mới, ký hiệu là S_{new}
  - Mã S_{new} được giao cho TestingAgent để chạy thử trên tập dữ liệu T. Kết quả được lưu vào biến $pass_{new}$.
  - ProfilingAgent cũng đem mã S_{new} chạy trên tập T để đo thời gian thực thi, lưu vào biến $per\ f_{new}$

- ***Cập nhật***:

  - Kết thúc mỗi vòng, hệ thống sẽ nối (Append) toàn bộ thông tin của vòng đó (r, S_{new}, $pass_{new}$, $per\ f_{new}$) vào Log.
  - Sau đó, hệ thống cập nhật các biến tạm: S_{prev} bằng S_{new}, $pass_{prev}$ bằng $pass_{new}$ và $per\ f_{prev}$ bằng $per\ f_{new}$ để làm đầu vào cho vòng lặp tiếp theo.

III. SO SÁNH PHƯƠNG PHÁP: MEP VÀ ASTRA FRAMEWORK

Mặc dù cả hai framework đều chia sẻ chung một mục tiêu là dùng LLM để tối ưu hóa GPU Kernel thông qua vòng lặp phản hồi để hiệu năng, cách thiết kế và cách tiếp cận bài toán giữa 2 bài lại khác biệt.

1. Cách tiếp cận:

a. MEP Framework:
Đối tượng: Tập trung vào các hotspot kernels trong các siêu ứng dụng HPC lớn như LAMMPS hoặc MISA-MD.
Cách làm: Phương pháp này sinh ra để giải quyết chi phí biên dịch lớn. LLM viết thêm code để để biến một đoạn kernel rỗng thành một chương trình chạy độc lập là MEP.

b. Astra Framework:
Đối tượng: Tập trung vào các kernel CUDA đã được viết sẵn và đang chạy trong các hệ thống phục vụ LLM thực tế như SGLang.
Cách làm: Astra lấy code có sẵn và tìm cách tối ưu hiệu năng bằng cách thay đổi cấu trúc thuật toán.

2. Kiến trúc:

- MEP Framework: Sử dụng một luồng LLM duy nhất: hoàn thiện mã nguồn, sinh ra các Candidate, đọc lỗi biên dịch để điều chỉnh, tự rút ra bài học cho vòng lặp sau. Yêu cầu LLM phải có khả năng suy luận tốt.
- Astra Framework: phân chia công việc cho 4 Agents là Testing Agent (kiểm thử), Profiling Agent (đo lường), Planning Agent (phân tích dữ liệu), và Coding Agent (code). Việc chia nhỏ vai trò của Astra giúp mỗi LLM tập trung vào đúng nhiệm vụ của mình, tránh bị hiện tượng như LLM hallucination.

3. Feedback:

- MEP Framework: Khi gặp lỗi, MEP dùng cơ chế *Automatic Error Repair* để tự sửa. Ví dụ, khi MEP có chiến lược chia block/grid hiệu quả, nó sẽ thêm vào Prompt của vòng lặp tiếp theo để giúp LLM đi theo hướng tối ưu đó.

- Astra Framework: Quyết định tối ưu được giao hoàn toàn cho Planning Agent và nó sẽ nhìn vào kết quả từ Testing và Profiling rồi mới mới giao cho Coding Agent để chạy.

4. Metrics:

a. MEP Framework:
Dữ liệu: Dùng một *Input Data Generator* để tự động sinh dữ liệu ngẫu nhiên sao cho thỏa mãn ràng buộc về thời gian và bộ nhớ
Đo lường: Vì chạy trên hệ thống HPC dễ bị nhiễu, MEP chạy N lần, cắt bỏ k kết quả cao nhất và k kết quả thấp nhất, sau đó tính **Trimmed Mean** để chọn ra bản code nhanh nhất.

b. Astra Framework:
Dữ liệu: Astra sử dụng các kích thước tensor được lấy từ thực tế chạy các mô hình lớn như LLaMA-7Bđể đảm bảo kernel tối ưu xong có thể áp dụng ngay vào thực tế.
Đo lường: Sử dụng **Geometric Mean** để tính toán tỷ lệ tăng tốc speedup trên toàn bộ tập test, giúp đánh giá công bằng hơn khi có các kết quả tăng hoặc giảm đột biến.

IV. Kết quả thực nghiệm của MEP Framework [1]

Kết quả trên bộ benchmark PolyBench: Bộ benchmark này chứa các bài toán tính toán ma trận và tương quan cơ bản.

Trên nền tảng NVIDIA GPU: Tốc độ tăng tốc trung bình sau khi tích hợp lại vào ứng dụng thì Integrated Speedup đạt mức **5.05x**. Còn phương pháp Direct LLM Optimization chỉ đạt 2.10x.

Name	Standalone	Integrated	Direct LLM Optimization
2MM	3.39	1.45	1.45
3MM	2.84	3.13	3.09
ADI	1.57	1.58	1.47
ATAX	2.19	1.06	1.05
BICG	7.14	7.22	1.00
CORR	14.64	14.27	5.42
COVAR	14.04	13.78	0.86
GEMM	2.75	3.48	3.63
GEMVER	7.10	7.23	3.59
GESUMMV	1.79	1.84	1.87
GRAMSCHM	6.55	5.89	1.73
SYR2K	3.70	3.74	1.02
SYRK	1.08	1.08	1.08
Average	5.29	5.05	2.10

Hình 7. PolyBench speedup trên NVIDIA

Kết quả trên Ứng dụng Siêu máy tính: Framework MEP đạt mức tăng tốc trung bình **1.25x**. phương pháp gọi LLM tối ưu trực tiếp Direct LLM gần như thất bại hoàn toàn vì tốc độ không đổi 1.00x do không có môi trường chạy thử và phản hồi lỗi.

V. Kết quả thực nghiệm của Astra Framework [2]

1. Kết quả hiệu năng tổng thể trên các kernel thực tế

Hệ thống Astra được thử nghiệm trực tiếp trên 3 kernel của SGLang thể hiện trong Hình 8. :

Index	Kernel Name	Computation
Kernel 1	`merge_attn_states_lse`	$\mathbf{V}_{\text{out}} = \frac{e^{S_a}\mathbf{V}_a + e^{S_b}\mathbf{V}_b}{e^{S_a} + e^{S_b}}$, $S_{\text{out}} = \log(e^{S_a} + e^{S_b})$
Kernel 2	`fused_add_rmsnorm`	$\mathbf{y} = \frac{\mathbf{x} + \mathbf{r}}{\sqrt{\frac{1}{D}\lVert\mathbf{x} + \mathbf{r}\rVert_2^2 + \varepsilon}} \odot \mathbf{w}$
Kernel 3	`silu_and_mul`	$\mathbf{out} = \text{SiLU}(\mathbf{x}) \odot \mathbf{g}$, $\text{SiLU}(z) = \frac{z}{1+e^{-z}}$

Hình 8. Các kernel thử nghiệm cho Astra

Correctness: 100% các kernel do Astra tối ưu hóa đều vượt qua bài kiểm tra tính đúng đắn, cho ra kết quả tương đương với code gốc của SGLang

Speedup: Sau 5 vòng lặp tối ưu bằng mô hình OpenAI o4-mini, hệ thống đạt mức tăng tốc trung bình **1.32x**. Cụ thể, tốc độ tăng lần lượt là 1.26x, 1.25x và cao nhất là 1.46x đối với kernel số 3.

Kernel	LoC-Base	LoC-Opt.	ΔLoC	Time-Base	Time-Opt.	**Speedup**	**Correct**
Kernel 1	124	232	+87%	31.4	24.9	1.26×	✓
Kernel 2	108	163	+50%	41.3	33.1	1.25×	✓
Kernel 3	99	157	+59%	20.1	13.8	1.46×	✓
Average	110	184	+64%	30.9	23.9	1.32×	✓

Hình 9. Kết quả thử nghiệm 3 Kernel

2. Kết quả khi chạy Multi-Agents và Single-Agent

Tài liệu đã thực hiện một phép so sánh trực diện giữa việc dùng 4 Agent phối hợp (Astra) và việc nhồi nhét mọi nhiệm vụ cho 1 Agent duy nhất:

Hiệu quả: Trong khi Multi-Agent đạt tốc độ **1.32x**, Single-Agent chỉ đạt mức tăng tốc trung bình rất khiêm tốn là **1.08x**.

Tác vụ phức tạp: Theo bài báo đánh giá, đối với kernel phức tạp nhất là Kernel 1, Single-Agent sinh code chạy chậm 0.73x do ***tự sinh ra*** tập dữ liệu test bị, dẫn đến việc đánh giá hiệu năng bị sai. Multi-Agent không bị nhờ có tác nhân Testing và Profiling độc lập.

VI. KẾT QUẢ ĐÁNH GIÁ CODE KERNEL CỦA CÁC LLM (DỰA TRÊN KERNELBENCH) [3]

Nghiên cứu KernelBench gồm 250 bài tập chia làm 3 cấp độ, nhằm đánh giá khả năng các mô hình AI hiện đại code GPU kernel. Các kết quả đánh giá được chia thành các khía cạnh sau:

1. One-shot Baseline:

LLM được cung cấp một ví dụ mẫu one-shot và yêu cầu viết code tối ưu.

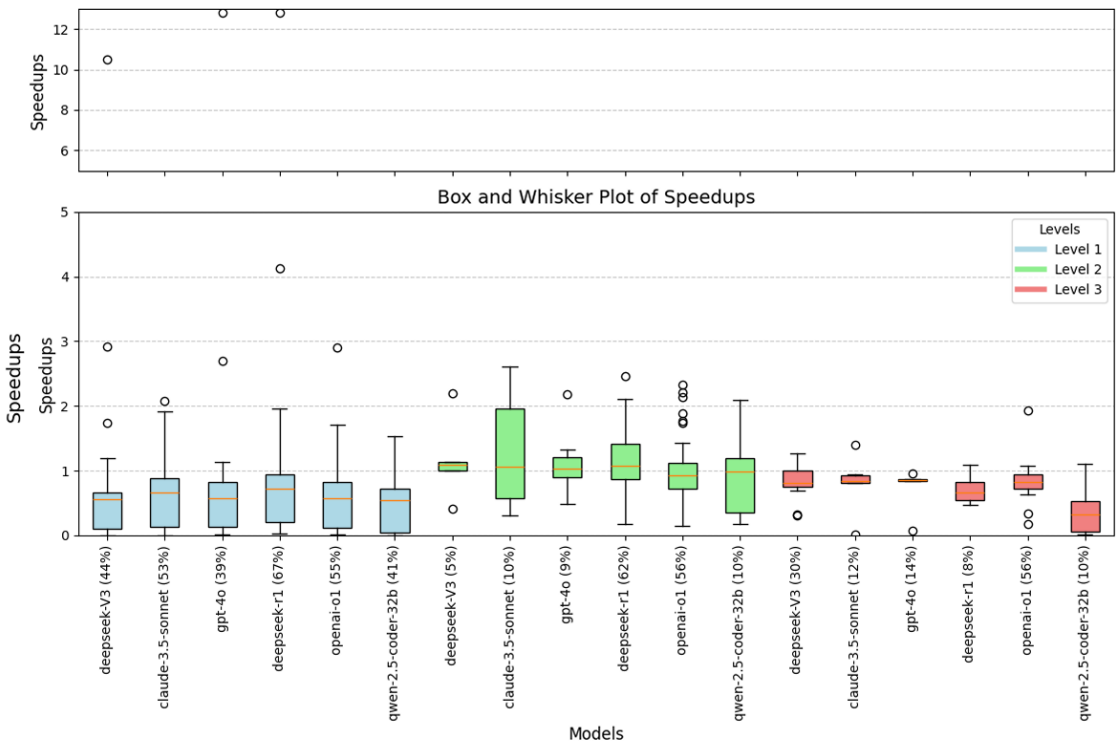
Kết quả chung: các LLM sinh ra code có tốc độ nhanh hơn baseline PyTorch Eager trong chưa tới 20% tổng số bài toán.

$\mathbf{fast}_1$ over:	PyTorch Eager			torch.compile		
KernelBench Level	1	2	3	1	2	3
GPT-4o	4%	5%	0%	18%	4%	**4%**
OpenAI o1	10%	24%	**12%**	28%	19%	**4%**
DeepSeek V3	6%	4%	8%	20%	2%	2%
DeepSeek R1	**12%**	**36%**	2%	**38%**	**37%**	2%
Claude 3.5 Sonnet	10%	7%	2%	29%	2%	2%
Llama 3.1-70B Inst.	3%	0%	0%	11%	0%	0%
Llama 3.1-405B Inst.	3%	0%	2%	16%	0%	0%

Hình 10. Các LLMs có speedup nhanh hơn Baseline PyTorch Eager trong chưa tới 20% tổng số bài toán

Các mô hình chuyên về reasoning có kết quả khá hơn. Cụ thể, Hình 10. Cho thấy DeepSeek-R1 dẫn đầu ở Level 2 về tính chuỗi toán tử với tỷ lệ $fast_1$ đạt 36%, trong khi OpenAI-o1 đạt 24%.

Tốc độ thực tế: Theo biểu đồ phân phối theo như Hình 11. thì LLM sinh ra code chạy đúng, nhưng median speedup của hầu hết các mô hình đều nhỏ hơn 1, tức là chạy chậm hơn cả code gốc của PyTorch.

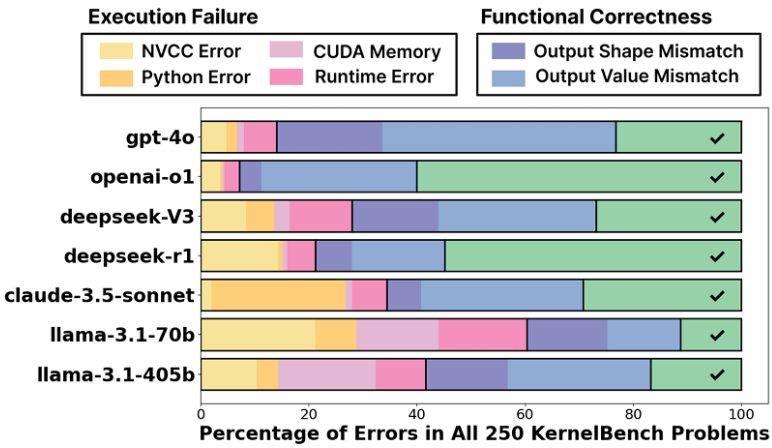


Hình 11. Box and Whisker Plot thể hiện Speedup của kernel so với Torch Eager mà chỉ chạy one-shot.

2. Error Analysis

Bài báo phân loại sự thất bại của LLM thành 2 nhóm:

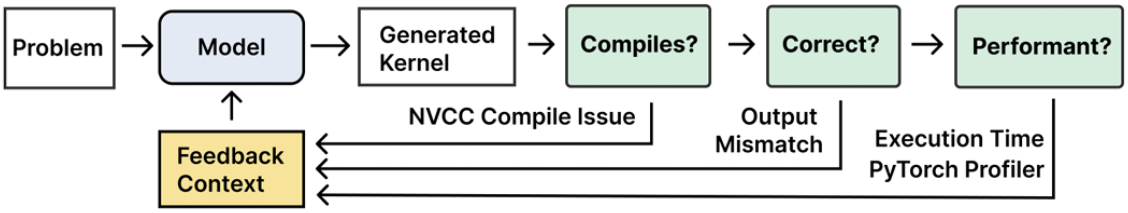
- **Execution Failures:** lỗi biên dịch NVCC, lỗi tràn bộ nhớ CUDA, lỗi runtime. Các mô hình o1 và R1 làm tốt hơn khi có tỷ lệ lỗi dưới 20%, các mô hình khác như Llama 3.1 bị lỗi thực thi trên 50%.
- **Functional Correctness:** sai kích thước tensor hoặc sai giá trị toán học Lưu ý, tất cả các mô hình gặp tỷ lệ thất bại khá cao ở loại lỗi này.



Hình 12. Phân loại các loại lỗi khi LLM sinh ra code GPU

3. Feedback Loop

Vì kết quả One-shot khá thấp, tài liệu đã thử nghiệm các kỹ thuật hỗ trợ LLM khi sinh code:



Hình 13. KernelBench framework cho phép LLMs biết được phản hồi

- **Repeated Sampling:** Việc cho LLM thử sinh code nhiều lần (ví dụ: k=100 lần) giúp tăng tỷ lệ sinh ra code tối ưu hơn. DeepSeek-V3 cải thiện $fast_1$ từ 4% lên 37% ở Level 2.
- **Iterative Refinement:** Hình 13. Thể hiện quy trình LLM được cung cấp phản hồi về lỗi biên dịch và dữ liệu thời gian chạy để tự sửa lỗi sau mỗi vòng lặp. DeepSeek-R1 ở Level 2 đã tăng điểm $fast_1$ từ 36% lên tới 72% sau 10 vòng lặp có phản hồi đầy đủ.

VII. KẾT LUẬN

Khảo sát này đã trình bày việc ứng dụng Mô hình ngôn ngữ lớn (LLM) trong tác vụ tối ưu hóa mã nguồn cho các ứng dụng AI và HPC. Qua việc phân tích ba nghiên cứu, chúng ta thấy rằng qua framework KernelBench, việc chỉ dùng LLM như một công cụ sinh code thôi là không đủ để đáp ứng tối ưu hiệu năng phần cứng. Đồng thời, điểm chung của cả phương pháp MEP và Astra là việc thiết lập một quy trình tối ưu hóa lặp lại dựa trên dữ liệu thực tế và điều này cũng đã được chứng minh một phần thông qua thử nghiệm Feedback Loop về Iterative Refinement của KernelBench khi các mô hình LLM cho kết quả cải thiện.

TÀI LIỆU THAM KHẢO

[1] GPU Kernel Optimization Beyond Full Builds: An LLM Framework with Minimal Executable Programs.

Truy cập: https://arxiv.org/abs/2512.22147

[2] Astra: A Multi-Agent System for GPU Kernel Performance Optimization

Truy cập: https://arxiv.org/abs/2509.07506

[3] KernelBench: Can LLMs Write Efficient GPU Kernels?

Truy cập: https://arxiv.org/abs/2502.10517